MICROCHIP TECHNOLOGY

Getting Started with Zephyr RTOS

PIC32WM BZ6 Curiosity

Macos

Table of Contents

Getting Started with ZephyrOS

ZephyrOS is an open source RTOS targeted towards embedded systems that includes community support for many Microchip development boards. This class will introduce an engineer to the coding environment, SDK, and debug tools available within the ZephyrOS ecosystem. Using hands-on examples, the engineer will gain experience with useful OS primitives and tasks, explore the hardware's Device Tree, build and deploy to target hardware.

Upon completion, you will:

- Learn how to install and navigate a ZephyrOS project
- Gain an understanding of the basic ZephyrOS commands and mechanics
- Create a new ZephyrOS project structure, ready for a clean application build
- Use Zephyr's `west` tool to build and debug project code for a given target
- Become familiar with the usage of built-in kernel APIs for message queues, semaphores, and tasks
- Gain insight into the ZephyrOS Devicetree system including sources and overlays

Prerequisites

The lab material assumes you have prior experience with:

- Any embedded RTOS (Real Time Operating System)
- C language programming

Conventions

Code blocks use the following conventions to mark changes and important lines:

Highlighted lines (yellow background) indicate lines that have been modified.

```
int a = 1;
int b = 2;
int c = a + b;
```

Green lines indicate newly added code.

```
int a = 1;
int b = 2;
int c = a + b;
int d = c * 2;
```

Red lines indicate code that should be deleted.

```
int a = 1;
int b = a / 0;
int c = a + 1;
```

Bold lines indicate a line of particular importance.

```
int a = 1;
void importantFunction(void);
```

Shell session blocks show the full prompt in a muted color so you can see which terminal you are in. The **Copy** button copies only the command, not the prompt. Selecting text manually also skips the prompt.

```
(.venv) $ west build -p always -b same54_xpro .
(.venv) $ west flash
```

```
uart:~$ kernel threads list
```

```
(.venv) PS C:\Users\you\zephyrproject> west build -p always -b same54_xpro .
```

ℹ Relevant information about a specific topic.

⚠ A critical detail that could cause issues if overlooked.

✔ A helpful suggestion or shortcut that can make your workflow easier.

Software Requirements

The following software is required for all labs.

- Visual Studio Code
    - **Serial Monitor** extension - install from the VSCode Extensions tab: marketplace.visualstudio.com
- Zephyr SDK and host tools

All needed software will be pre-installed for the in-person class. For future reference, the completed lab code can be found at: github.com/sarpuser/getting-started-with-zephyr

Install required software

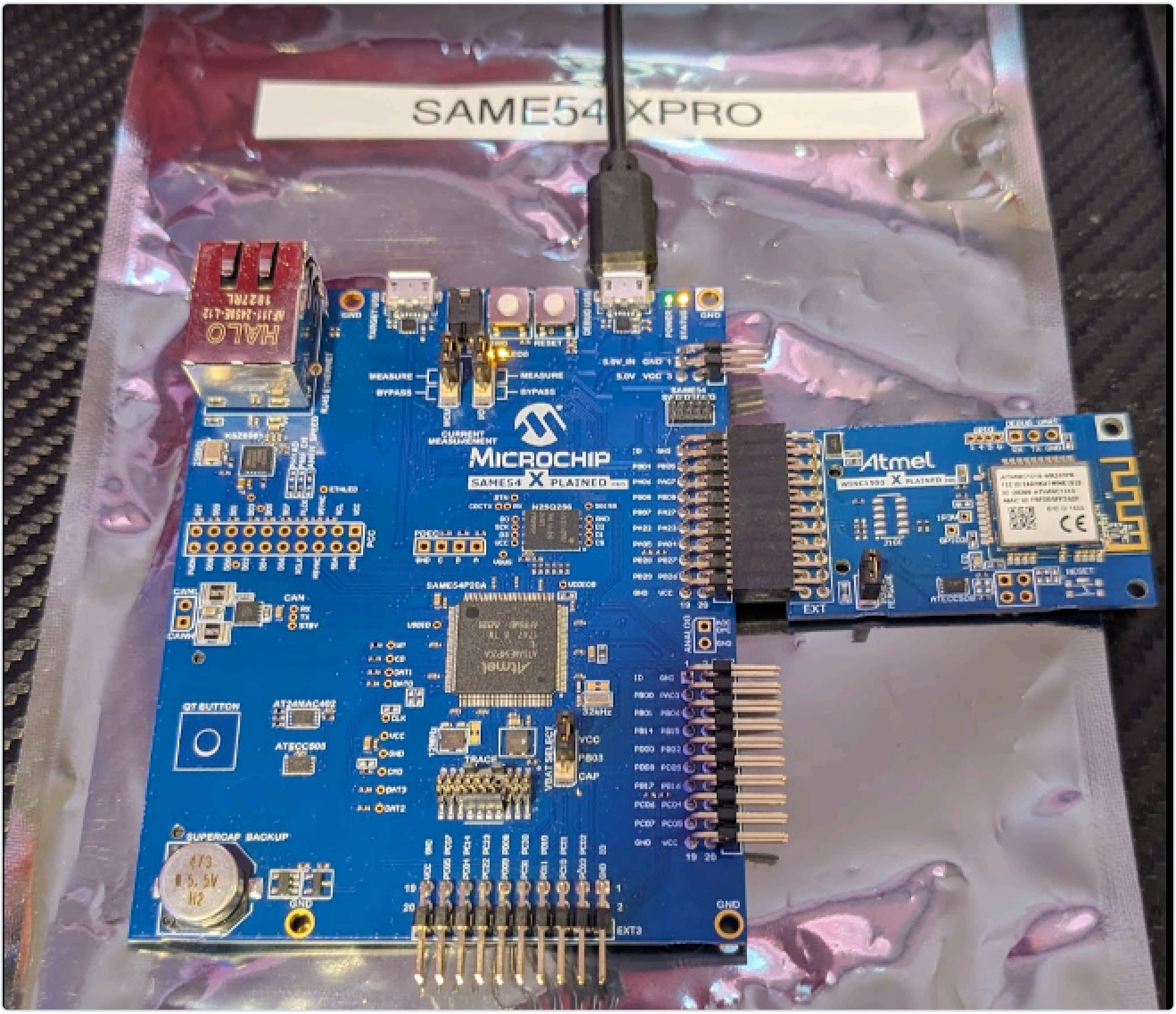
See Appendix A: Host PC Setup Guide

PIC32BZ6 - Board Overview

Hardware Requirements

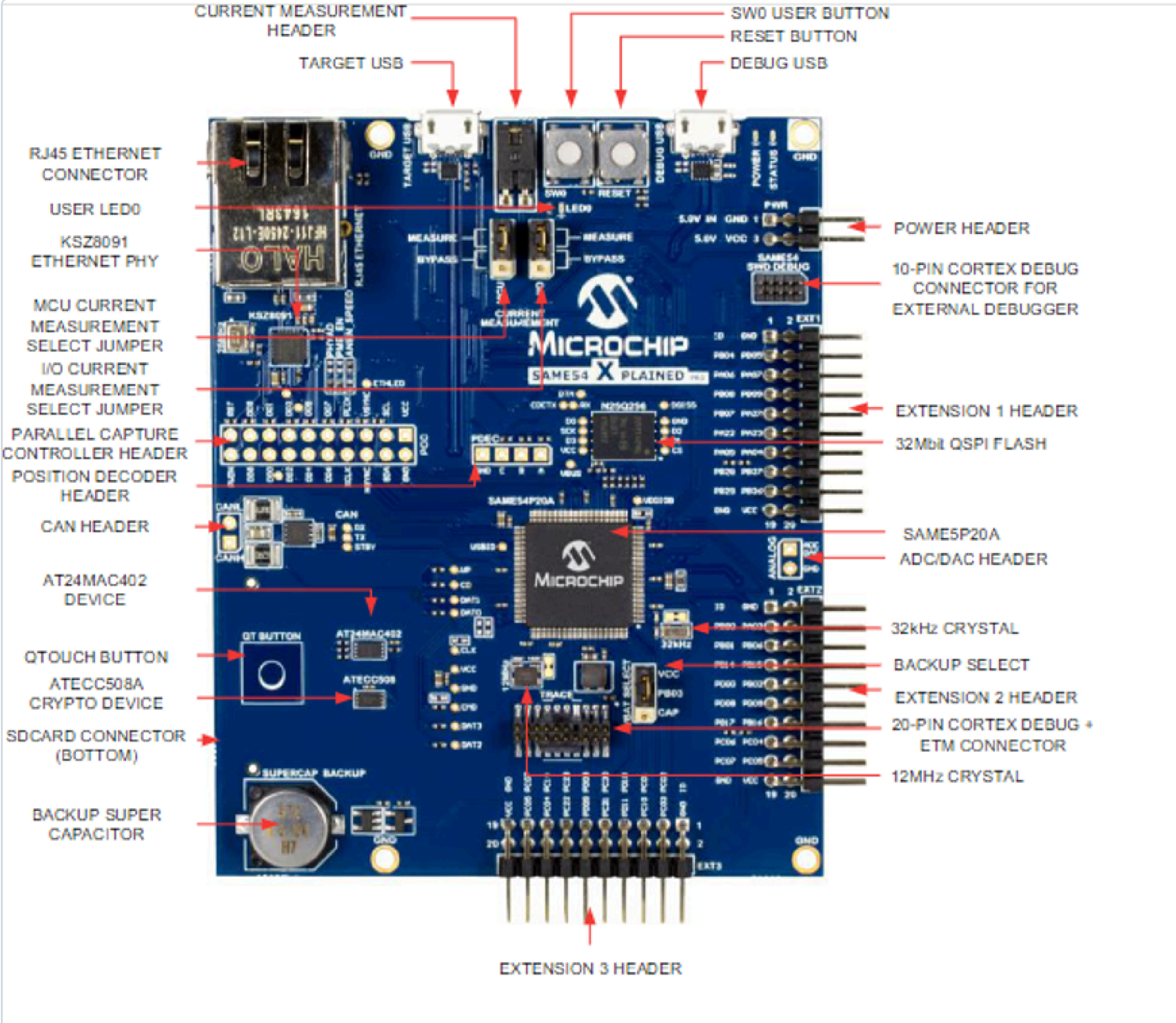
Here is the list of hardware needed to complete the lab:

- The SAME54 XPLAINED PRO
- 1× Micro-USB cable
- ATWINC1500-XPRO Extension Board



The SAM E54 Xplained Pro evaluation kit is a hardware platform for evaluating the ATSAME54P20A microcontroller (MCU). Supported by the Studio integrated development platform, the kit provides easy access to the features of the ATSAME54P20A and explains how to integrate the device into a custom design. This board is supported by MPLAB integrated tools and MPLAB Harmony.

The ATWINC1500-XPRO is an extension board to the Xplained Pro evaluation platform. The ATWINC1500-XPRO extension board allows you to evaluate the ATWINC1500 low cost, low power 802.11 b/g/n Wi-Fi network controller module.



More information:

microchip.com - ATSAME54-XPRO

Additional Software Setup

This board requires additional software setup. Complete all steps below before proceeding to the labs.

Switching to the Zephyr4Microchip Repo

Zephyr support for this board has not yet been released to the main Zephyr repo. To that end, we need to use the Microchip fork of Zephyr. This fork contains support for the latest Microchip boards and will eventually be mainlined into Zephyr. Since you have already completed the Zephyr Getting Started Guide, run the following commands from your Zephyr project directory to switch to the Microchip fork:

```
$ git -C zephyr remote set-url origin https://github.com/Zephyr4Microchip/zephyr
```

```
$ git -C zephyr fetch
```

```
$ git -C zephyr checkout mchp_pic32cx_v420
```

```
$ source .venv/bin/activate
```

```
(.venv) $ west update
```

```
(.venv) $ west blob fetch hal_microchip
```

Downloading the Custom OpenOCD Binary

Adding new board support to the upstream OpenOCD release takes time. Microchip provides a customized OpenOCD binary that supports the latest boards ahead of upstream. Clone the repository into your Zephyr workspace:

```
$ cd ~/zephyrproject
$ git clone https://github.com/MicrochipTech/openOCD-wireless
```

Installing Zephyr SDK v0.17.4

> **⚠ The Zephyr Getting Started Guide installs the latest SDK, which is currently v1.0.1. This version is **not compatible** with the Microchip Zephyr fork used in this course. You must install SDK v0.17.4 alongside it.**

Run the following commands from your Zephyr project directory. Once installed, the build system will automatically select v0.17.4.

```
$ source .venv/bin/activate
```

```
(.venv) $ west sdk install --version 0.17.4 -t arm-zephyr-eabi
```

> **✔ The `-t arm-zephyr-eabi` flag installs only the ARM cross-compiler toolchain, which targets the ARM Cortex-M4 processor on this board. Without it, `west sdk install` downloads cross-compilers for every supported target architecture (~4 GB). The host machine architecture (your laptop's CPU) is detected automatically regardless of this flag.**

Lab 1: Build and Deploy a Sample Application

Purpose

In this lab, the student will build and deploy their first sample application, then copy the files from this sample application into their own project directory, making a skeleton ZephyrOS project that will be used for the rest of these labs.

Overview

This lab will begin by opening VSCode and a terminal pane. You will then initiate your Python Virtual Environment (VENV) and use this terminal to issue “west” commands to build and flash code to your PIC32WM BZ6 Curiosity device. You will also open a second terminal window and use SerialMonitor to view serial output from your device.

Procedure

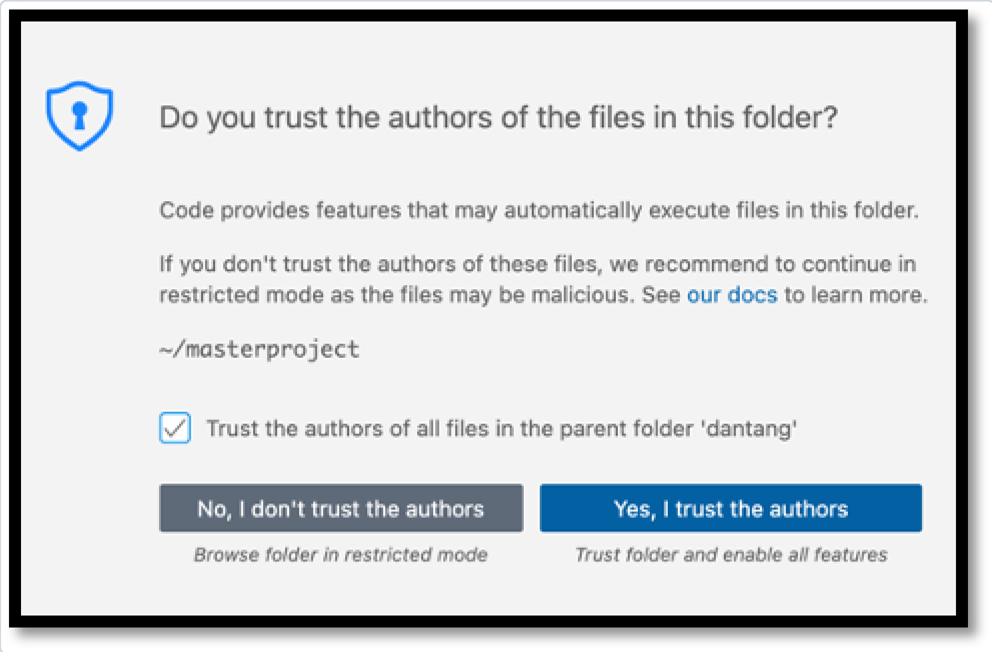
Step 1.1: Build a sample application using *west*

1.1.1: Launch a VSCode window by clicking on the VSCode icon or by changing directory to your project directory and typing:

```
$ cd ~/zephyrproject
```

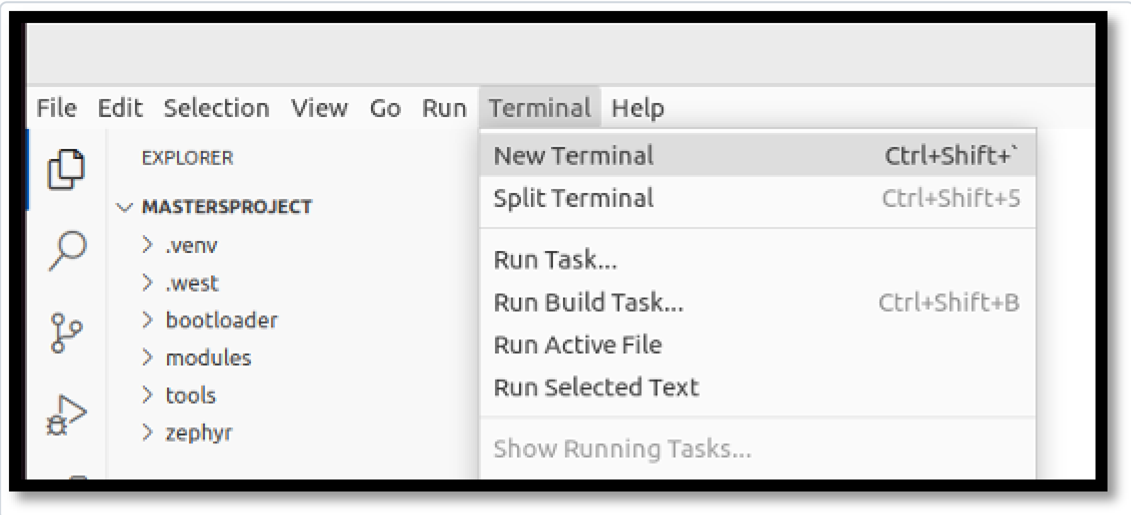
```
$ code .
```

1.1.2: Opening VSCode will result in a confirmation window asking you to trust the folder, choose the button labeled "Yes, I trust the authors"



1.1.3: When VSCode is loaded, enable autosave by navigating to File → AutoSave

1.1.4: Open a terminal pane in VSCode by navigating to Terminal → New Terminal



1.1.5: The `west` tool is a Python executable that can be installed globally on a system, or locally within a Python Virtual Environment (venv). Our West install is in one such venv. Each time a new terminal window is opened, the environment should be sourced with the following command:

```
$ source ~/zephyrproject/.venv/bin/activate
```

When complete, your terminal prompt will include the name of your virtual environment:

```
(.venv) $
```

You can learn more about Python Virtual Environments here: docs.python.org/3/tutorial/venv.html

1.1.6: From within your terminal (and `.venv`), build your first sample project targeting the PIC32WM BZ6 Curiosity board:

```
(.venv) $ west build -p always -b pic32wm_bz6204_curiosity zephyr/samples/basic/blinky
```

ℹ **Observe the output and success messages from your build, which may resemble the following:**

```
←[92m-- west build: building application
[1/117] Generating include/generated/zephyr/version.h
-- Zephyr version: 3.7.99 (C:/Users/C75166/mastersproject/zephyr), build: v3.7.0-3346-g45727b5fe141
[117/117] Linking C executable zephyr\zephyr.elf
Memory region         Used Size  Region Size  %age Used
           FLASH:       14664 B         1 MB      1.40%
             RAM:        4352 B       256 KB      1.66%
        IDT_LIST:          0 GB        32 KB      0.00%
Generating files from C:/Users/C75166/mastersproject/build/zephyr/zephyr.elf for board: same54_xpro
```

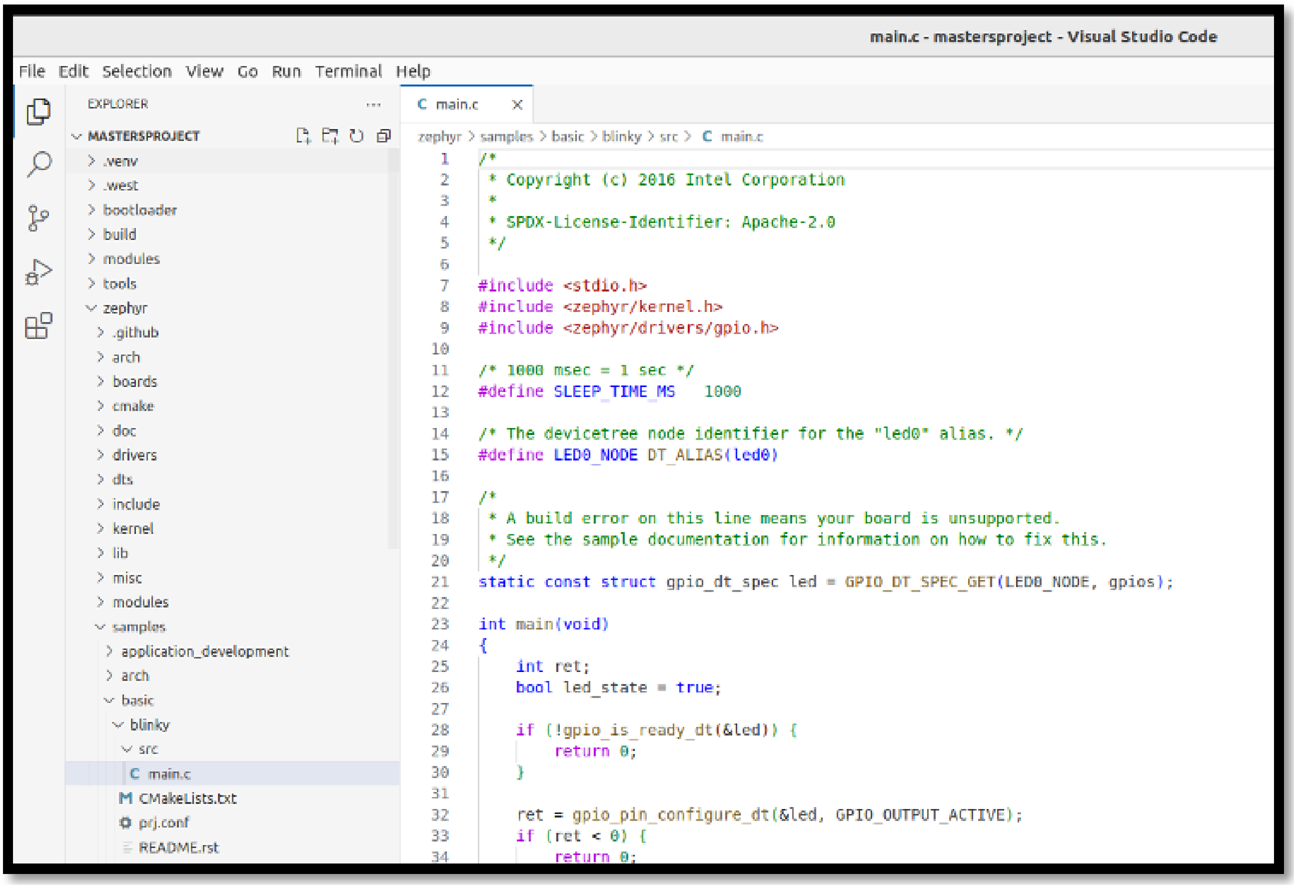
Step 1.2: Flash the sample application to your target device using *west*

1.2.1: To flash your PIC32WM BZ6 Curiosity with the compiled code, connect your PIC32WM BZ6 Curiosity board with your MicroUSB cable connected to the "Debug USB" port of your target, allow your Operating System to find and mount the device, then type:

```
(.venv) $ west flash --openocd openOCD-wireless/prebuilt_binaries/macos/bin/openocd
```

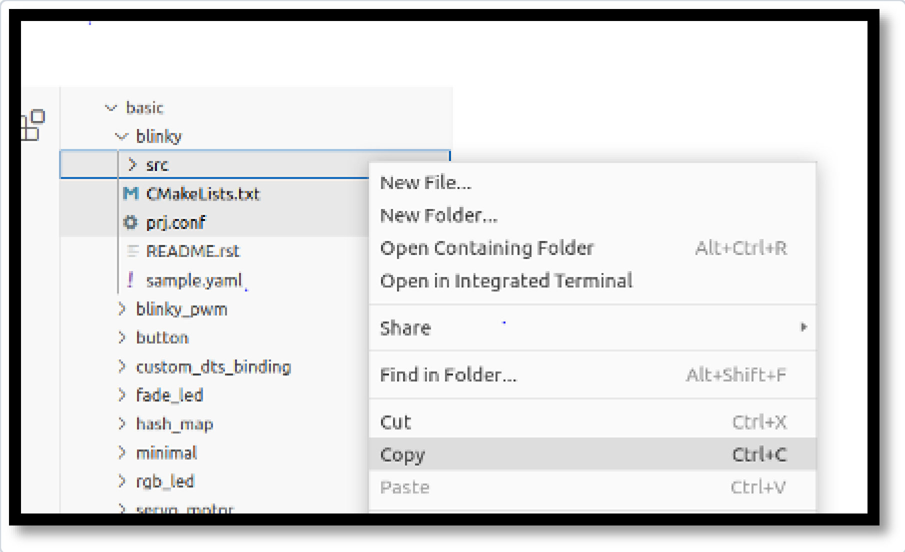
> ℹ **The device is now programmed and LED0 should be flashing at a rate of 1Hz.**

1.2.2: Observe the code of the sample application by using the VSCode Navigation pane to expand `zephyr/samples/basic/blinky/src` and opening `main.c` in the main panel. Many of these code elements will be discussed within the remainder of this course.



Step 1.3: Populate a new project tree, then build and deploy your custom project

1.3.1: It is often easiest to use collateral from a sample project to begin creating your own project. Right-Click/Copy the `blinky` folder from this sample project and paste them to the root of the project. Then, rename the folder you just pasted to `application` You can delete the `README.rst` and `sample.yml` files.



You can also use your terminal window and run the following commands:

```
(.venv) $ cp -r zephyr/samples/blinky application
```

1.3.2: Build this Blinky code in its new location:

```
(.venv) $ west build -p always -b pic32wm_bz6204_curiosity application
```

1.3.3: Close any existing VSCode tabs for `main.c`, then reopen `main.c` from `./application/src/`. Edit the blink time in `main.c` to change the sleep time between blinks:

`src/main.c` Line 12:

```
#define SLEEP_TIME_MS 500
```

1.3.4: Rebuild your code with the custom changes included. If you do not need a clean build and your target options haven't changed since your last build, you can simply type:

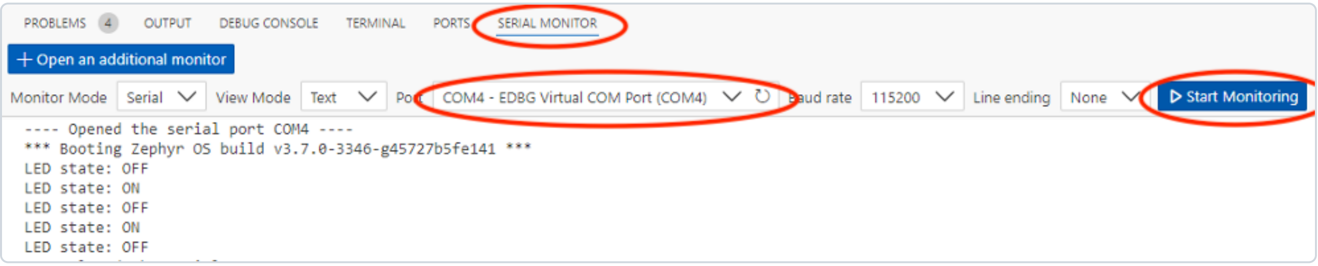
```
(.venv) $ west build
```

> ✔ **You can skip the full `west build -p always…` command for incremental builds. West is generally**
> smart enough to detect code changes and recompile as necessary before flashing. If newly added code doesn't seem to be running as intended, try a pristine build again.

1.3.5: Flash this newly built code to your PIC32WM BZ6 Curiosity device and observe the new blink frequency:

```
(.venv) $ west flash --openocd openOCD-wireless/prebuilt_binaries/macos/bin/openocd
```

1.3.6: In a new terminal window (Terminal → New Terminal), select the "SERIAL MONITOR" tab and connect to the serial port (Start Monitoring) of the device (EDBG Virtual COM Port) to see print statements from our `main()` loop.



Results

Congratulations! You have successfully built and flashed your first Zephyr projects!

Summary

This lab introduced you to VSCode, *west*, the Zephyr sample application repository, and SerialMonitor - these are all of the tools you'll need to use for the rest of these labs.

Lab 2: Threads and Message Queue

Purpose

Create your first tasks in Zephyr, exploring different options for creating tasks and passing in data and communication structures that your tasks might need to operate.

Overview

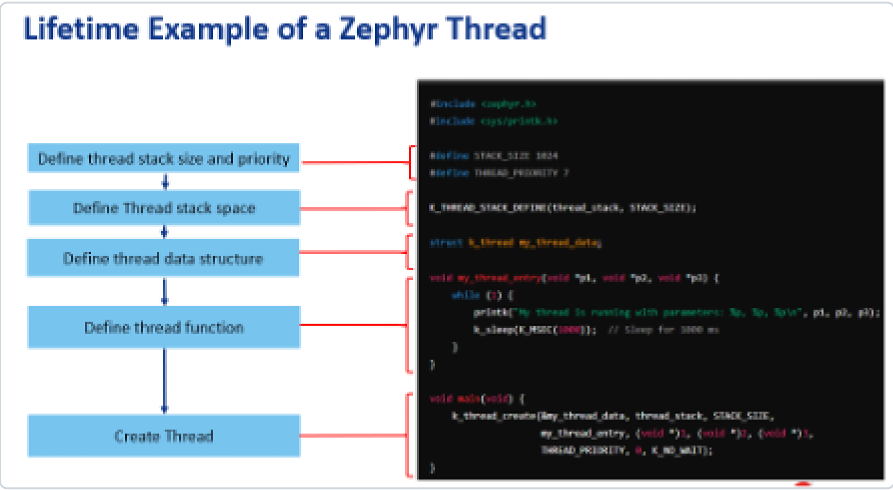
Zephyr allows the engineer to quickly and effectively create a task to be run by the scheduler system. Tasks can be created at compile time or at runtime and assigned priorities to help determine importance to the system.

When creating tasks, it’s important to minimize “busy wait” or blocking functions where the scheduler may have difficultly taking control from the task.

We’ll also create a message queue that will be used to deliver data from one task to another, and learn how to allow the tasks to access the message queue.

Procedure

In the following steps, we’ll complete the following high level actions in order to run a thread in Zephyr:



> ℹ **Along the way in this lab, you may notice some *compiler warnings* when you build. These are due to the interative nature of this lab. By the time Lab 2 is complete all compiler warnings will**
>
> be cleared!

Step 2.1: Create a Producer Thread

2.1.1: In the `application/src/` folder, create a new file called `producer.c`, adding the following content:

```
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>
#include "producer.h"

void producerThread(void *inLed, void*, void*) {
      struct gpio_dt_spec *toggleLED = (struct gpio_dt_spec*)inLed;
      bool led_state = true;
      int counter = 0;
      uint32_t data = 0;

    while (1) {
        gpio_pin_toggle_dt(toggleLED);
        led_state = !led_state;
        k_msleep(SLEEP_TIME_MS);
    }
}
```

> ℹ **Notice that this function takes three void pointers and returns void. All Zephyr tasks must match this prototype. Void pointers can be cast to a useful type within the function as shown in this**
>
> example. It's a good idea to create objects at a higher level and pass them in via this technique (called dependency injection) if multiple tasks need to share resources.

> ℹ **Zephyr provides a special delay function `k_msleep()` that allows the individual task to wait for the specified timeout (in milliseconds) while allowing the scheduler to service other tasks within the system. If you need a delay, always use this sleep function (or one of its siblings – `k_sleep()`, `k_usleep()`, `k_yield()`, or `k_busy_wait()` [if you truly need a blocking delay])**

2.1.2: : In the `application/src/` folder, create a file called `producer.h` with a define for `SLEEP_TIME_MS` and a prototype for `producerThread()`:

```
#define SLEEP_TIME_MS     100
void producerThread(void* inLed, void*, void*);
```

2.1.3: From the top of `main.c`, remove the `#define SLEEP_TIME_MS 500` to prevent a redefine. Further down in `main()`, delete the instantiation of `bool led_state = true`; since we moved this part of the application to `producer.c`.

```
#include <zephyr/drivers/gpio.h>

#define SLEEP_TIME_MS 500

...

int main(void) {
        int ret;
        bool led_state = true;

        if (!gpio_is_ready_dt(&led)) {
                return 0;
        }
```

2.1.4: Update `CMakeLists.txt` to add `producer.c` as a build target and expose the `src/` directory as an include path:

```
target_sources(app PRIVATE src/main.c src/producer.c)
target_include_directories(app PRIVATE src)
```

> **ℹ `target_include_directories(app PRIVATE src)` tells CMake to add the `src/` folder to the**
>
> compiler's header search path. This is what allows `#include "producer.h"` (and later `#include "consumer.h"`) to resolve without an explicit relative path prefix. **Notably, this is relative to the `CMakeLists.txt` file, hence why we are not using `application/src/`.**

> **✔ If you prefer to store your include files elsewhere in the file tree, you are free to do so. Just be**
>
> sure to update this line in `CMakeLists.txt` accordingly. In any case, the linker will now be able to find any `.h` files that are added to this directory.

2.1.5: Replace the `while(1)` loop body in `main.c` with a simple sleep so that `main()` yields the CPU once threads are running:

```
while(1) {
    k_msleep(SLEEP_TIME_MS);
}
```

2.1.6: In `main.c`, include `producer.h` by adding the following line towards the top of the file:

```
#include <stdio.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>
#include "producer.h"
```

2.1.7: Still in `main.c`, use the following compile time macro to create a stack space for the task by adding the following code before `main()`;

```
/* The devicetree node identifier for the "led0" alias. */
#define LED0_NODE DT_ALIAS(led0)

/*
 * Zephyr Thread defines
*/

#define STACKSIZE 1024
#define PRIORITY 7
K_THREAD_STACK_DEFINE(producerThreadstack_area, STACKSIZE);
struct k_thread producerThread_data;
```

2.1.8: Initialize the task in `main()` right before the `while(1){}` loop:

```
    ret = gpio_pin_configure_dt(&led, GPIO_OUTPUT_ACTIVE);
        if (ret < 0) {
                return 0;
        }

    k_tid_t producer_tid = k_thread_create(&producerThread_data,
                               producerThreadstack_area,
                               K_THREAD_STACK_SIZEOF(producerThreadstack_area),
                               producerThread,
                               (void*)&led, (void*)NULL, (void*)NULL,
                               PRIORITY, 0, K_NO_WAIT);

    while (1) {
        k_msleep(SLEEP_TIME_MS);
    }
```

2.1.9: Build and flash your new code using `west flash`. The LED should toggle every 100ms. (10Hz)

> **✔ You can of course continue to type the full `west build -p always...` command, but it is often**
>
> quicker to just run `west flash`. West is generally smart enough to know when code has changed and recompile as necessary, then flash the resulting compiled code. If you ever find that newly added code doesn't seem to be running as intended, try a pristine build again.y.

Step 2.2: Create a Consumer Thread

2.2.1: In the `application/src/` folder, create a new file called `consumer.c`, adding the following content:

```
#include <zephyr/kernel.h>
#include "consumer.h"

void consumerThread(void*, void*, void*){
    uint32_t data = 0;

    while (1) {
        k_msleep(1000); // 'Magic Number' OK - We'll replace shortly
        printk("(consumer) Data: %d\n", data);
        data++;
    }
}
```

2.2.2: In the `application/src/` folder, create a filed called `consumer.h` with a prototype for `consumerThread()`:

```
void consumerThread(void*, void*, void*);
```

2.2.3: In `main.c`, follow the steps you learned previously to instantiate and run your new task with its own stack space and TID:

2.2.3.1: Include the header file for your new task at the top of `main.c`

```
#include <zephyr/drivers/gpio.h>
#include "producer.h"
#include "consumer.h"
```

2.2.3.2: Add the `consumer.c` file to the `CMakeLists.txt` `target_sources`

```
target_sources(app PRIVATE src/main.c src/producer.c src/consumer.c)
```

2.2.3.3: In `main.c`, create a stack area in memory for your thread (you can use the same `#define STACKSIZE` that you used for Producer Thread) and create a new variable to hold your thread data:

```
/*
 * Zephyr Thread defines
 */
#define STACKSIZE 1024
#define PRIORITY 7
K_THREAD_STACK_DEFINE(producerThreadstack_area, STACKSIZE);
struct k_thread producerThread_data;
K_THREAD_STACK_DEFINE(consumerThreadstack_area, STACKSIZE);
struct k_thread consumerThread_data;
```

2.2.3.4: Call `k_thread_create()` with your new thread's information, being sure to check that you are passing expected void pointers – in this case, `(void*)NULL` for all three members

```
    k_tid_t producer_tid = k_thread_create(&producerThread_data,
                            producerThreadstack_area,
                            K_THREAD_STACK_SIZEOF(producerThreadstack_area),
                            producerThread,
                            (void*)&led, (void*)NULL, (void*)NULL,
                            PRIORITY, 0, K_NO_WAIT);
    k_tid_t consumer_tid = k_thread_create(&consumerThread_data,
                            consumerThreadstack_area,
                            K_THREAD_STACK_SIZEOF(consumerThreadstack_area),
                            consumerThread,
                            (void*)NULL, (void*)NULL, (void*)NULL,
                            PRIORITY, 0, K_NO_WAIT);
    while (1) {
        k_msleep(SLEEP_TIME_MS);
    }
```

2.2.3.5: Build and flash your code, examining the output in Serial Monitor. You should see output from both tasks appear in your console.

Step 2.3: Create a Message Queue

2.3.1: In `main.c` – just before the `main()` function, declare a new global variable to hold the Message Queue and a buffer that will hold the number of queue items we wish to include (in this case, our buffer will hold a maximum of 4 32-bit queue items):

```
static const struct gpio_dt_spec led = GPIO_DT_SPEC_GET(LED0_NODE, gpios);

struct k_msgq consumerQueue;
char taskCommsBuffer[4 * sizeof(uint32_t)];

int main(void) {
    ...
```

2.3.2: Initialize the Message Queue in `main()` before your thread creation by calling `k_msgq_init()`:

```
int main (void) {

    ...

    k_msgq_init(&consumerQueue, taskCommsBuffer, sizeof(uint32_t), 4);

    k_tid_t producer_tid = k_thread_create(...
```

2.3.3: Update `producer.c/h` prototypes of `producerThread()` to take the MessageQueue pointer as its second argument, as so:

```
void producerThread(void *inLed, void* inMsgQueue, void*)
```

2.3.4: Allow `producer.c:producerThread` to use this Message Queue by casting the void pointer in `producerThread()` just below where inLED is cast to `struct gpio_dt_struct`:

```
void producerThread(void *inLed, void* inMsgQueue, void*) {
       struct gpio_dt_spec *toggleLED = (struct gpio_dt_spec*)inLed;
       struct k_msgq *notifyMsgQueue = (struct k_msgq*)inMsgQueue;
```

2.3.5: In `producer.c`, add the following block of code within the `while(1){}` loop to periodically place data into the message queue:

```
    led_state = !led_state;
    if(counter++>=10) {
        printk("(producer) Putting %d into message queue\n", data);
        k_msgq_put(notifyMsgQueue, &data, K_NO_WAIT);
        data++;
        counter = 0;
    }

    k_msleep(SLEEP_TIME_MS);
```

> ℹ **The data we've chosen to pass in this example isn't particularly useful, but you can easily**
>
> imagine an ADC collecting data and passing a rolling average value to other threads periodically

2.3.6: In `main.c`, update your `k_thread_create()` call to take the `&consumerQueue` pointer (created in step 1) as its second `(void*)` argument to match your updated `producerThread()` prototype

```
producer_tid = k_thread_create(&producerThread_data,
                producerThreadstack_area,
                K_THREAD_STACK_SIZEOF(producerThreadstack_area),
                producerThread,
                (void*)&led, (void*)&consumerQueue, (void*)NULL,
                PRIORITY, 0, K_NO_WAIT);
```

2.3.7: Similarly, update `consumer.c/h`:

2.3.7.1: Accept `(void*)inMsgQueue` as its second argument for `consumerThread()` prototype:

```
void consumerThread(void*, void* inMsgQueue, void*)
```

2.3.7.2: In `consumer.c` cast the incoming `(void*) inMsgQueue` pointer for use within the `consumerThread` function

```
void consumerThread(void*, void* inMsgQueue, void*) {
    struct k_msgq *notifyMsgQueue = (struct k_msgq*)inMsgQueue;
    uint32_t data = 0;

        while (1) {
        ...
```

2.3.7.3: In `consumer.c` replace the `k_msleep()` command in `consumerThread` with a command to get a queue item in the message queue:

```
    while (1) {
        k_msleep(1000);
        k_msgq_get(notifyMsgQueue, &data, K_FOREVER);
        printk("(consumer) Received data: %d\n", data);
    }
```

ℹ `K_FOREVER` seems like a long time. If your thread needs to wait that long for data, something

may have gone wrong. In the real world, you may want to time bound this function and raise an error if the timer expires without a new queue entry showing up as expected.

2.3.7.4: In `consumer.c` remove the `data++`; incrementor from `consumerThread()`, as data is now being collected from the message queue and doesn't need to be internally incremented

```
    while (1) {
        k_msgq_put(notifyMsgQueue, &data, K_NO_WAIT);
        printk("(consumer) Data: %d\n", data);
    }
}
```

2.3.7.5: In `main.c` update your `k_thread_create()` call to take your `&consumerQueue` pointer as its second `(void*)` argument to match your updated `consumerThread()` prototype

```
consumer_tid = k_thread_create(&consumerThread_data,
                consumerThreadstack_area,
                K_THREAD_STACK_SIZEOF(consumerThreadstack_area),
                consumerThread,
                (void*)NULL, (void*)&consumerQueue, (void*)NULL,
                PRIORITY, 0, K_NO_WAIT);
```

2.3.8: Build and flash your code, observing in SerialMonitor that the value of "counter" in `producerThread()` is now delivered to `consumerThread()` via the shared message queue, and `consumerThread` now prints out the value whenever a new queue item is received.

Results

Congratulations! You have completed Lab 2. Your device is now running two Zephyr OS tasks with a Message Queue to deliver data from one to the other!

Summary

Learning to manage tasks and inter-task communication are important steps to building your RTOS application. In this Lab you created two tasks and learned to pass data between them. There are many other ways to pass data among tasks including Semaphores, Events, Pipes, and more. Browse the ZephyrOS API docs for more information and usage!

Lab 3: Add a Shell and Winc1500 to your project

Purpose

In this lab, the student will enable the Zephyr shell over UART, inspect running threads with built-in shell commands, name threads for easier debugging, add a Device Tree overlay to expose the ATWINC1500 Wi-Fi module over SPI, and use the network shell to scan for and connect to a Wi-Fi access point.

Overview

Lab 3 builds on the two-thread project from Lab 2. First you will enable the Zephyr shell and explore the system at runtime. Next you will write a Device Tree overlay that wires the ATWINC1500-XPRO extension board to the PIC32WM BZ6 Curiosity's SPI peripheral. Finally you will add the necessary Kconfig options and use the network shell to bring up a Wi-Fi connection.

Procedure

Step 3.1: Enable and Explore the Zephyr Shell

3.1.1: Open `prj.conf` and add the following line to enable the Zephyr shell subsystem:

```
CONFIG_SHELL=y
```

3.1.2: Open `src/producer.c` and `src/consumer.c` and remove the `printk` calls in the thread functions so the shell prompt is not obscured by continuous output:

In `producer.c`:

```
        if(counter++>=10) {
            printk("(producer) Putting %d into message queue\n", data);
            k_msgq_put(notifyMsgQueue, &data, K_NO_WAIT);
            data++;
            counter = 0;
        }
```

In `consumer.c`:

```
    while (1) {
        k_msgq_get(notifyMsgQueue, &data, K_FOREVER);
        printk("(consumer) Received data: %d\n", data);
    }
```

3.1.3: Build and flash the updated firmware:

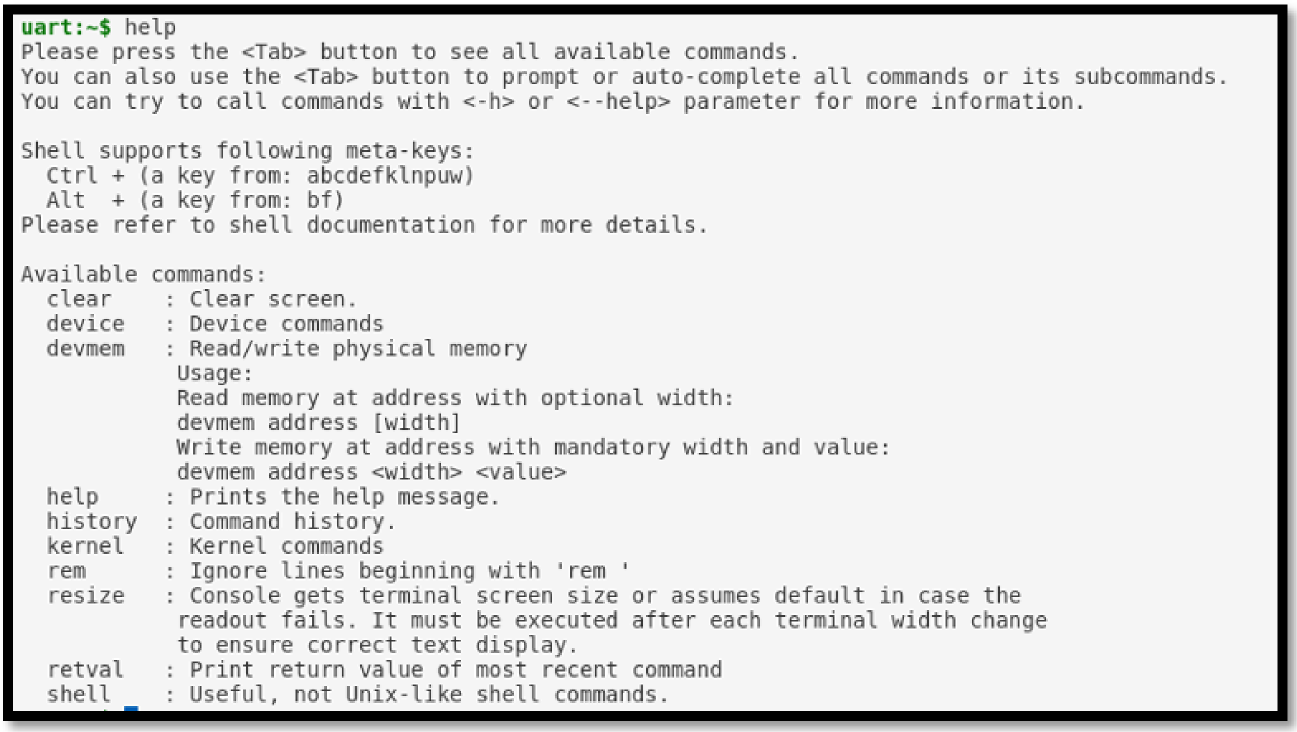
```
(.venv) $ west build -p always -b pic32wm_bz6204_curiosity application
```

```
(.venv) $ west flash --openocd openOCD-wireless/prebuilt_binaries/macos/bin/openocd
```

3.1.4: Open the Serial Monitor. To use the interactive shell you must configure line endings to CRLF (both send and receive). Set this option in the Serial Monitor settings panel before connecting.

Once connected you should see the `uart:~$` prompt. Type `help` to list all registered shell modules and their top-level commands:

```
uart:~$ help
```



```
uart:~$ help
Please press the <Tab> button to see all available commands.
You can also use the <Tab> button to prompt or auto-complete all commands or its subcommands.
You can try to call commands with <-h> or <--help> parameter for more information.

Shell supports following meta-keys:
  Ctrl + (a key from: abcdefklnpuw)
  Alt  + (a key from: bf)
Please refer to shell documentation for more details.

Available commands:
  clear    : Clear screen.
  device   : Device commands
  devmem   : Read/write physical memory
             Usage:
             Read memory at address with optional width:
             devmem address [width]
             Write memory at address with mandatory width and value:
             devmem address <width> <value>
  help     : Prints the help message.
  history  : Command history.
  kernel   : Kernel commands
  rem      : Ignore lines beginning with 'rem '
  resize   : Console gets terminal screen size or assumes default in case the
             readout fails. It must be executed after each terminal width change
             to ensure correct text display.
  retval   : Print return value of most recent command
  shell    : Useful, not Unix-like shell commands.
```

3.1.5: List all registered Zephyr devices by typing the following command at the shell prompt:

```
uart:~$ device list
```

```
device list
devices:
- eic@40002800 (READY)
- gpio@41008180 (READY)
- gpio@41008100 (READY)
- gpio@41008080 (READY)
- gpio@41008000 (READY)
- random@42002800 (READY)
- sercom@41012000 (READY)
- sercom@43000000 (READY)
```

Observe the output - every driver that was compiled into the image and successfully initialised will appear here.

3.1.6: List all active Zephyr threads and their current state, priority, and stack usage:

```
uart:~$ kernel threads
```

```
Scheduler: 623 since last call
Threads:
 0x20000090
        options: 0x0, priority: 7 timeout: 0
        state: pending, entry: 0x55c9
        stack size 1024, unused 824, usage 200 / 1024 (19 %)

 0x20000148
        options: 0x0, priority: 7 timeout: 1
        state: suspended, entry: 0x5587
        stack size 1024, unused 832, usage 192 / 1024 (18 %)

*0x20000200 shell_uart
        options: 0x0, priority: 14 timeout: 0
        state: queued, entry: 0x1c81
        stack size 2048, unused 928, usage 1120 / 2048 (54 %)

 0x200005d8 sysworkq
        options: 0x1, priority: -1 timeout: 0
        state: pending, entry: 0x4bb5
```

Notice that the producer and consumer threads appear with auto-generated numeric names. The next step will make them easier to identify.

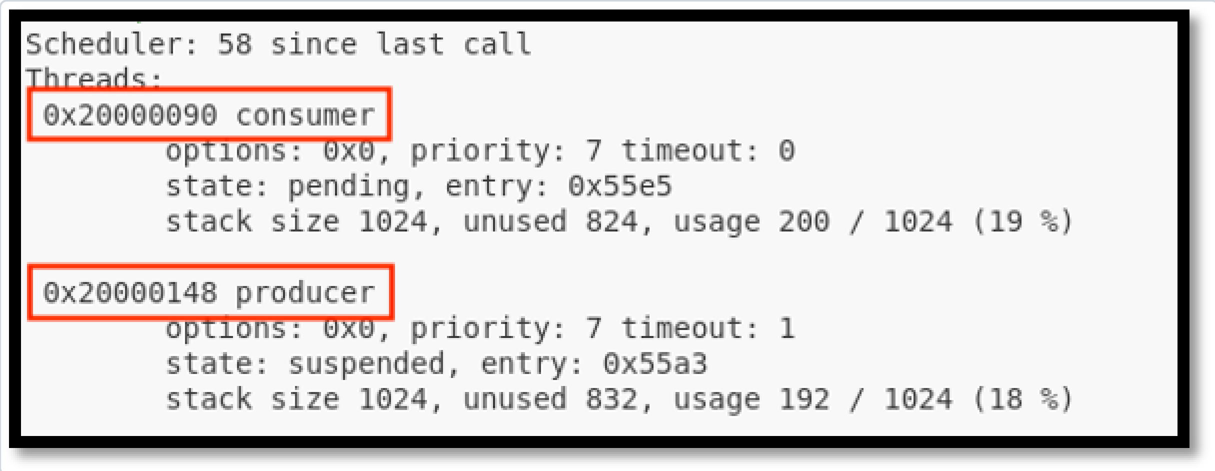
3.1.7: Open `main.c` and add the following two calls immediately after the `k_thread_create` calls for the producer and consumer threads:

```
k_thread_name_set(producer_tid, (const char *)"producer");
k_thread_name_set(consumer_tid, (const char *)"consumer");
```

3.1.8: Rebuild, flash, and rerun `kernel threads` in the shell. The producer and consumer threads should now appear with their human-readable names.

```
(.venv) $ west build
```

```
(.venv) $ west flash --openocd openOCD-wireless/prebuilt_binaries/macos/bin/openocd
```



```
Scheduler: 58 since last call
Threads:
0x20000090 consumer
        options: 0x0, priority: 7 timeout: 0
        state: pending, entry: 0x55e5
        stack size 1024, unused 824, usage 200 / 1024 (19 %)

0x20000148 producer
        options: 0x0, priority: 7 timeout: 1
        state: suspended, entry: 0x55a3
        stack size 1024, unused 832, usage 192 / 1024 (18 %)
```

3.1.9: CHALLENGE - Stack Usage Analysis

> **ℹ While looking at the `kernel threads` output, examine the **unused stack** column for your**
>
> **ℹ While looking at the `kernel threads` output, examine the **unused stack** column for your**
>
> producer and consumer threads. The `STACKSIZE` macro in `main.c` is currently set to 1024 bytes for both threads.
>
> Consider: is 1024 bytes the right size? If the unused stack is very large you are wasting RAM. If it is very small the thread is at risk of a stack overflow. Try reducing `STACKSIZE` for one or both threads and observe whether the firmware still builds and runs correctly. What is the minimum safe stack size for each thread?

Step 3.2: Add a Device Tree Overlay for the ATWINC1500

The ATWINC1500-XPRO extension board connects to the PIC32WM-BZ6204 Curiosity board via the XPRO header (J900). The table below shows the signal mappings for this connector.

WINC1500 Signal	XPRO Pin	PIC32WM-BZ6204 Pin
WAKE	3	RPD4 (portd 4)
RESET	4	RPB15 (portb 15)
CHIP_EN	6	RPD2 (portd 2)
IRQ	9	RPB0 (portb 0)
SPI CS	15	RPB1 (portb 1)
SPI MOSI	16	RPB3 (portb 3)
SPI MISO	17	RPA4 (porta 4)
SPI SCK	18	RPE5 (porte 5)

> **ℹ SPI MOSI, MISO, and SCK are managed automatically by the SERCOM4 SPI peripheral through**
> the `sercom4_spi_xpro` pinctrl node defined in the board support package — you do not need to list them as individual GPIOs in the overlay.

3.2.2: Create a `boards/` directory inside your project root and create the overlay file:

```
(.venv) $ mkdir -p boards
```

```
(.venv) $ touch boards/pic32wm_bz6204_curiosity.overlay
```

3.2.3: Open `boards/pic32wm_bz6204_curiosity.overlay` and add the following Device Tree content to configure SERCOM4 as an SPI bus and attach the WINC1500 as a device on that bus:

```
/ {
    aliases {
    };
};

&sercom4 {
    compatible = "microchip,sercom-g1-spi";
    dipo = <0>;
    dopo = <2>;
    #address-cells = <1>;
    #size-cells = <0>;
    cs-gpios = <&portb 1 GPIO_ACTIVE_LOW>;
    pinctrl-0 = <&sercom4_spi_xpro>;
    pinctrl-names = "default";
    dmas = <&dmac 4 12>, <&dmac 5 13>;
    dma-names = "rx", "tx";
    status = "okay";

    sercom4_cs0_winc1500: WINC1500@0 {
        compatible = "atmel,winc1500";
        reg = <0>;
        spi-max-frequency = <12000000>;
        irq-gpios    = <&portb 0  GPIO_ACTIVE_LOW>;
        reset-gpios  = <&portb 15 GPIO_ACTIVE_LOW>;
        enable-gpios = <&portd 2  GPIO_ACTIVE_HIGH>,
                       <&portd 4  GPIO_ACTIVE_HIGH>;
        status = "okay";
    };
};
```

ℹ `dipo = <0>` places MISO on SERCOM4 PAD[0] (RPA4) and `dopo = <2>` places SCK on PAD[1]

(RPE5) and MOSI on PAD[3] (RPB3). The two `enable-gpios` entries control CHIP_EN (RPD2) and WAKE (RPD4) respectively.

Appendix A: Host PC Setup Guide

ZephyrOS can be installed and used on most recent builds of Windows, macOS, and Linux. In order to re-create this lab on your own host PC, install the required dependencies as follows.

Step 1: Follow the Zephyr Getting Started Guide

This step is common to all platforms. Follow the official Zephyr documentation to install the Zephyr SDK, Python dependencies, and toolchain for your operating system:

https://docs.zephyrproject.org/latest/develop/getting_started/index.html

Step 2: Install OpenOCD

> ⚠ **OpenOCD must be installed separately from the Zephyr SDK. Even if your Zephyr environment is fully configured, flashing and debugging with OpenOCD will fail unless OpenOCD is installed on your host and available on your system PATH.**

Install OpenOCD via Homebrew:

```
brew install openocd
```

Reference: OpenOCD Debug Host Tools - Zephyr Docs

Step 3: Install VSCode and the Serial Monitor Extension

Download and install Visual Studio Code for your platform:

https://code.visualstudio.com/download

Once installed, open VSCode and navigate to the **Extensions Marketplace** (Ctrl+Shift+X / Cmd+Shift+X). Search for and install the **Serial Monitor** extension.

Appendix B: Debug your project in VSCode

Purpose

Explore options for debugging a target within a compiled application.

Overview

Debugging is an important tool for embedded development. Zephyr and VSCode offer several options for connecting a debugger to the target to see code flow and interrogate variables at different points of your project's operation.

Procedure

1. These line numbers assume you have just completed Lab 1. If you are at another point in your lab work, please either revert to the end of Lab 1 or adjust line numbers and variables to match your current code state.

2. Ensure that your latest code is compiled and flashed to your board, then launch the debugger:

```
(.venv) $ cd ~/zephyrproject && west debug
```

3. After this command completes, the console will leave you in a `(gdb)` prompt. Depending on your terminal window size, you may need to hit the **Return** key a few times to reach this prompt. There are several ways to interact with GDB (type `help` to learn more), but one useful option is to use the Text User Interface:

```
(gdb) tui enable
```

4. Add a breakpoint to your GDB session, then continue operation until the breakpoint is hit:

```
(gdb) break main.c:44
```

```
(gdb) continue
```

5. The device will now run until the breakpoint at `main.c` line 44 is reached. When operation is broken at that point, view the value of a variable:

```
(gdb) display led_state
```

```
(gdb) c
```

Observe the displayed value of the variable `led_state` on subsequent loops of the main loop, and verify that this variable matches the state of LED0 on your board:

GDB TUI view showing source code at breakpoint on main.c line 44 with led_state values displayed

6. Exit GDB and return to your venv command prompt:

```
(gdb) exit
```

If prompted, choose `y` to quit the active session.

Appendix C: Flashing the WINC1500 Firmware

Introduction

Just as in MPLABX/Harmony applications, it is important for the firmware version on the WINC1500 and the driver version in software to be in sync. In Zephyr, the driver version used is based on version **19.5.2**, so the WINC1500 firmware version should match.

> ⚠ **While firmware can be updated from other platforms, this procedure is generally accomplished most easily from a MS Windows computer. Updating from macOS or Linux is subject to change and is outside the scope of this supplement. If you are on Mac/Linux, you will need access to a Windows machine for this step, or you may use the USB UART dongle approach described below.**

Procedure

1. Download ASF standalone version **3.35.1**, which includes WINC1500 Firmware version 19.5.2, from the link below:

https://ww1.microchip.com/downloads/Secure/en/DeviceDoc/asf-standalone-archive-3.35.1.54.zip

If other versions are needed, the full list of previous ASF versions is available for download here.

2. Unzip the download to a folder, then navigate to the following path within it:

```
asf-standalone-archive-3.35.1.54/xdk-asf-
3.35.1/common/components/wifi/winc1500/firmware_update_project
```

3. Connect your WINC1500 module to **EXT1** on a *Supported Board*, then connect your supported board to your host computer over the **Debug USB** port.

At the time of writing, supported boards are:

- SAMW25 XPRO
- SAM4S XPRO
- SAMD21 XPRO
- SAMG53 XPRO
- SAMG55 XPRO
- SAML21 XPRO
- SAML22 XPRO
- SAMR21 XPRO

A sample program can be created for other boards as well by following the guidance of an existing program. Alternatively, you can use a USB UART dongle connected to the Debug UART port on the WINC1500 XPlained Pro extension board or on your custom hardware. Check the Application Note in the `doc/` folder entitled

WINC_Devices_Integrated_Serial_Flash_Download_Procedure.pdf for more information regarding this process.

4. On a Windows machine, inside the `firmware_update_project` folder, run the `.bat` file that matches your chosen supported board and wait while the firmware is updated automatically. This process may take up to **2 minutes**.

When complete, you will see a command window with the following output:

WINC1500 firmware update command window showing certificates written and downloading ends successfully

5. Scroll up within the command window output and verify that your firmware has been set to **v19.5.2** as seen below:

WINC1500 firmware version confirmation output showing Firmware ver 19.5.2 Svnrev 14274

WINC1500 firmware update has now been completed, and you may continue with this lab manual.

> ✔ **You may need to restore your WINC1500 firmware version at a later time. To do so, simply follow these same steps using a different standalone ASF version from the download link in Step 1.**

Appendix D: Using the Zephyr4Microchip Repo

The Zephyr4Microchip repo is a fork of the main Zephyr repository that contains support for the latest Microchip dev boards. These contributions will eventually be mainlined to the main Zephyr repo. For more information, take a look at the Zephyr4Microchip website.

Using the Zephyr4Microchip repo *before* running `west init`

Instead of running `west init` to download the main Zephyr repo, it is possible to directly download the Microchip4Zephyr repo. Go to the folder for your zephyr project and run the following in your terminal:

```
$ source ~/zephyrproject/.venv/bin/activate
```

```
(.venv) $ west init -m https://github.com/Zephyr4Microchip/zephyr.git --mr mchp_pic32cxbz_v420
```

```
(.venv) $ west blob fetch hal_microchip
```

Switching to the Zephyr4Microchip Repo *after* running `west init`

If you already ran `west init` and downloaded the regular Zephyr repo, you need to change the git remote and run `west update`. Run the following commands from your Zephyr project directory:

```
$ git -C zephyr remote set-url origin https://github.com/Zephyr4Microchip/zephyr
```

```
$ git -C zephyr fetch
```

```
$ git -C zephyr checkout mchp_pic32cx_v420
```

```
$ source .venv/bin/activate
```

```
(.venv) $ west update
```

```
(.venv) $ west blob fetch hal_microchip
```

Appendix E: Setting Up OpenOCD for PKOB-Equipped Boards

Some newer Microchip development boards use a PKOB (PICkit On Board) programmer/debugger instead of the EDBG interface. The PKOB is not yet supported by the upstream OpenOCD release. Two setup steps are required to use these boards with Zephyr:

1. Clone the Microchip custom OpenOCD repository
2. Flash the PKOB with CMSIS-DAP firmware so that OpenOCD can communicate with it

> ✎ **Once the PKOB is running CMSIS-DAP firmware, MPLAB X will not be able to detect the board. To use MPLAB X again, restore the original firmware. See [Switching Back to MPLAB Firmware](#switching-back-to-mplab-firmware) below.**

Step 1: Cloning the Custom OpenOCD Repository

Adding new board support to the upstream OpenOCD release takes time. Microchip provides a customized OpenOCD binary that supports the latest boards ahead of upstream. Clone the repository into your Zephyr workspace:

```
$ cd ~/zephyrproject
$ git clone https://github.com/MicrochipTech/openOCD-wireless
```

The cloned repository contains both the OpenOCD binaries and the PKOB firmware files used in the next step.

Step 2: Flashing CMSIS-DAP Firmware onto the PKOB

Installing pycmsisdapswitcher

Activate your Zephyr virtual environment and install the `pycmsisdapswitcher` tool:

```
$ source ~/zephyrproject/.venv/bin/activate
```

```
(.venv) $ pip install pycmsisdapswitcher
```

Switching to CMSIS-DAP Firmware

With your board connected via USB, run the following from your Zephyr workspace directory to flash the CMSIS-DAP firmware:

```
(.venv) $ cd ~/zephyrproject
(.venv) $ pycmsisdapswitcher --action switch --target=evalboard --source=openOCD-wireless/pkob4-cmsis_dap-switcher/pkob4_app_cmsis-dap.hex --fwtype=cmsis
```

> ✎ **The firmware switching process can be inconsistent. If the command fails, disconnect and reconnect the board and try running it again.**

Once the switch is successful, reconnect the board. OpenOCD and `west flash` will now be able to communicate with the board through the PKOB.

Switching Back to MPLAB Firmware

To use MPLAB X with the board again, restore the original PKOB firmware:

```
(.venv) $ cd ~/zephyrproject
(.venv) $ pycmsisdapswitcher --action switch --target=evalboard --source=openOCD-wireless/pkob4-cmsis_dap-switcher/pkob4_app.hex --fwtype=mplab
```

> ✎ **As with the CMSIS-DAP switch, if the command fails, disconnect and reconnect the board and try again.**

After a successful switch, reconnect the board — MPLAB X will detect it as normal.